



B.TECH II YEAR I SEM --- BACKEND WEB TECHNOLOGIES

(Short Question and Answers)

UNIT-1

SHORT QUESTION AND ANSWERS:

1.Explain briefly about Routing?

Routing in Express refers to how an application's endpoints (URIs) respond to client requests. It matches incoming HTTP methods (like GET, POST, PUT, DELETE) and URL paths to specific backend handler functions.

Syntax:

`app.METHOD(PATH, HANDLER);`

Where:

- **app:** Represents an instance of an Express application.
- **METHOD:** Represents an HTTP method like GET, POST, PUT, DELETE.
- **PATH:** Defines the endpoint (route) where the request will be handled.
- **HANDLER:** A function that executes when the route is accessed.

Ex: `app.get('/', (req, res)`

2. What is mean by Server?

Servers are crucial in the backend architecture. They manage requests from the client side, process them, and return the appropriate response. This could involve querying a database, executing server-side scripts, or integrating with third-party services. Servers act as the bridge between the user and the application's data, ensuring that requests are handled efficiently and securely.

3.What is mean by Backend development?

The backend development, often referred to as the server-side, is the backbone of any web application. It is responsible for the server, application, and database that work together to make sure the frontend works as it should. The backend processes the business logic, database interactions, authentication, and much more.

4.What is the importance of Express.js?

Express.js is a popular framework for building web applications on top of Node.js. It provides a robust set of features and tools that make it easy to develop scalable and maintainable web applications.

Express.js is used to:

- Receive requests from users
- Send responses back to users
- Create websites
- Create APIs and Manage different URLs (routes)

5.Explain briefly about Node.js.

1. Node.js is a free, open-source, cross-platform JavaScript runtime environment that allows developers to execute JavaScript code outside of a web browser.
2. Node.js is neither a programming language nor a development framework; it is an environment that hosts and executes JavaScript code.
3. It is built on Google Chrome's high-performance V8 engine, which compiles JavaScript text directly into fast machine code.
4. Node.js uses an event-driven design. Instead of waiting for a file read or database query to finish, it moves to the next task immediately, making it incredibly lightweight and efficient.

6.What is the purpose of handling HTTP requests in web applications?

Purposes of Handling HTTP Requests

- Data Retrieval (GET requests) Allows clients to fetch resources such as web pages, images, or data from a database.
- Data Submission (POST requests) Enables users to send information to the server, like submitting a form, uploading a file, or posting a comment.
- Updating Resources (PUT/PATCH requests) Used to modify existing data on the server, such as editing a profile or updating settings.
- Deleting Resources (DELETE requests) Lets clients remove data from the server, like deleting an account or removing a post.
- Session & Authentication Management Requests often carry cookies, tokens, or headers that help maintain user sessions and verify identity.
- Dynamic Content Delivery Servers process requests, run business logic, and return customized responses (e.g., personalized dashboards).

LONG QUESTION AND ANSWERS:

1.Explain Backend Development. Discuss its role in web application architecture?

Backend development involves building the server-side "brain" of a website that manages user data, authentication, and database communication.

Backend development is focused on constructing and maintaining the server-side part of web applications. Backend development involves taking care of *database interaction*, *user authentication*

and authorization, server logic, and API integrations. It provides a seamless user experience, faster loading time, and secure handling of data.

Simply put, the backend, often referred to as the server-side, is the backbone of any web application. It is responsible for the server, application, and database that work together to make sure the frontend works as it should. The backend processes the business logic, database interactions, authentication, and much more.

Role in web application architecture

1. Processing the Request-Response Cycle: When a user clicks a button, the frontend sends an HTTP request. The backend receives this request, processes the underlying intent, communicates with the data layer, and returns an appropriate status code (like 200 OK or 404 Not Found) along with the requested payload, typically in JSON format.

2. Executing Business Logic: The backend enforces the operational rules of an organization. For instance, on an e-commerce platform, frontend code displays a checkout form, but the backend verifies item stock availability, calculates regional sales tax, and executes the actual transaction.

3. Data Management and Persistence: Web servers lack permanent memory; when a server restarts, temporary variables disappear. The backend orchestrates secure data transactions using Relational Database Management Systems (RDBMS) like PostgreSQL or NoSQL options like MongoDB to ensure data survives application lifecycles.

4. API Development and Integration: The backend connects different technical systems together. Developers use tools like Express or Django to build APIs. These APIs let the frontend load data and connect the app to outside services, like credit card processors or text-messaging systems.

5. Security, Authentication, and Identity Management: The backend acts as a digital guard for user data. It handles safe logins, saves passwords securely, and tracks who is logged in. It also checks all incoming information to block hackers and keep the app safe.

6. Scalability and Optimization: As traffic volume expands, backend architects scale systems horizontally by setting up load balancers to split heavy traffic across multiple application nodes. They also optimize speed by using in-memory databases to cache frequent queries and deploy message and to process non-blocking background tasks.

2.Explain Server Architecture in detail and discuss its major features?

Express.js follows a simple architecture where the **Client** sends a request, the **Express Server** processes it, and then sends a response back.

Server Architecture in Express.js

Express.js follows a simple Client → Server → Route → Response architecture.

1. Client

A client sends an HTTP request to the server.

Examples:

- Browser
- Mobile App
- Postman
- Frontend Application

Client



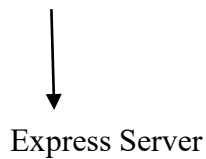
2. Express Server

The Express server receives the request and decides what to do with it.

```
const express = require('express');
```

```
const app = express();
```

Client



3. Middleware Layer

Middleware functions execute before the request reaches the route.

Common tasks:

- Logging
- Authentication
- Validation
- Parsing JSON

```
app.use(express.json());
```

```
app.use((req, res, next) => {  
  console.log("Request received");  
  next();  
});
```

Client



4. Routing Layer: Routes determine which function should handle the request.

```
app.get('/users', (req, res) => {  
  res.send("Users Data");  
});
```

Client



Middleware



Route Handler

5. Business Logic Layer

This layer contains the actual application logic.

Examples:

- Calculate results
- Process data
- Validate user input

6. Database Layer

The application may communicate with a database.

Examples:

- MongoDB
- MySQL
- PostgreSQL

Client



Middleware



Route



Database

7. Response

The server sends data back to the client.

Client



Response

Example:

```
import express from 'express';  
const app = express();
```

```
app.use(express.json());
```

// Middleware

```
app.use((req, res, next) => {  
  console.log("Request received");  
  next();  
});
```

// Route

```
app.get('/users', (req, res) => {
```

// Business Logic

```
  const users = [  
    { id: 1, name: "Ali" },  
    { id: 2, name: "Sara" }  
  ];
```

// Response

```
  res.json(users);
```

```
});
```

```
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

Output:



What happens?

1. Browser requests /users
2. Express server receives request
3. Middleware executes
4. Route /users matches
5. Route handler prepares data
6. Response is sent to browser

This is the basic server architecture of Express.js: **Client** → **Middleware** → **Route** → **Controller** → **Database** → **Response**.

3. Explain CommonJS and ESModules in detail

Common JS and ES Modules

A module is a self-contained unit of code that encapsulates related functionality. This allows developers to organize their code into logical, reusable components. Such components can be easily imported and used in other parts of their project.

Node.js supports two module systems for organizing and sharing code. These are

- **CommonJS modules (CJS)**
- **ES modules (ESM)**

CommonJS modules (CJS)

CommonJS is the module system that Node.js adopted as its default before ES Modules existed. It uses the **require()** method to load other modules and **module.exports** to expose values from a file.

Every **require()** call is synchronous; the file is read and executed immediately before the program continues. CommonJS loads the entire exported object synchronously. It copies values upon export rather than binding directly to them.

Example:

JS main.js

JS math.js

```
JS main.js > ...
1  const { add, subtract, multiply } = require('./math.js');
2
3  console.log(add(5, 3));
4  console.log(subtract(5, 3));
5  console.log(multiply(5, 3));
6
```

JS main.js

JS math.js

```
JS math.js > ...
1  const add = (a, b) => a + b;
2  const subtract = (a, b) => a - b;
3  const multiply = (a, b) => a * b;
4
5  module.exports = {add, subtract, multiply};
```

Output:

```
Node.js v24.13.1
PS D:\NODEJS> node main.js
8
2
15
PS D:\NODEJS>
```

ES modules (ESM)

In ES modules exports and require keywords are not used. Instead, modules use the export and import keywords to export and import variables, functions, etc.

Example:

JS main.js

JS math.js

```
ESPROJ > JS main.js
1  import { add, subtract } from './math.js';
2
3  console.log(add(5, 3));
4  console.log(subtract(5, 3));
```

JS main.js

JS math.js

```
ESPROJ > JS math.js > ...
1  export const add = (a, b) => a + b;
2  export const subtract = (a, b) => a - b;
3
```

Output:

```
PS D:\> CD NODEJS
PS D:\NODEJS> CD ESPROJ
PS D:\NODEJS\ESPROJ> node main.js
8
2
PS D:\NODEJS\ESPROJ>
```

CommonJS vs ES Modules

Feature	CommonJS	ES Modules
Import	require()	import
Export	module.exports	export
Loading	Synchronous	Asynchronous
Default in old Node.js	Yes	No
Used in modern JavaScript	Less	More
Browser Support	No	Yes

4. Briefly explain about Error Handling in Express.js.

Error handling in Express refers to the process of managing errors that occur during the execution of a request. When an error occurs in a route or middleware, Express provides mechanisms to catch these errors and prevent the application from crashing. Error-handling middleware allows developers to:

- Log errors for debugging.
- Send structured error responses to users.
- Maintain application stability even when issues arise.

Good error handling makes an application more reliable and user-friendly, improving both developer efficiency and the user experience.

Types of Error Handling in Express

There are multiple ways to handle errors in Express; few of them are:

1. Built-in Error Handling (The Default)

Express comes with a default, built-in error handler that catches any synchronous errors that occur in your route handlers or middleware. Its job is to handle errors when **you don't create your own error-handling middleware**. If you don't add your own error handler, Express uses its default one. If you throw an error synchronously, Express will automatically catch it, set the HTTP status to 500 (Internal Server Error), and write a stack trace to the response (in development mode).

2. Custom Error-Handling Middleware

Instead of relying on the built-in error handler, you can create a custom error middleware. This lets you log errors and send user-friendly responses. Express recognizes this as an error-handling middleware because it defines **exactly four arguments (err, req, res, next)**. It must be placed at the very bottom of your middleware stack, after all your routes.

3. Synchronous Error Handling

Synchronous error handling means handling errors that occur immediately while Express is executing code line by line in a route handler. Express can catch these errors automatically and pass them to the error-handling middleware. If you don't create your own error-handling middleware, Express uses its built-in (default) error handler.

4. Asynchronous Error Handling in Express.js

Asynchronous Error Handling in Express.js is the process of detecting, catching, and handling errors that occur during asynchronous operations (such as reading files, querying databases, or making API requests) without crashing the application. Instead of allowing the application to fail, these errors are passed to Express's error-handling middleware using `next(err)`, where they can be handled and an appropriate response can be sent to the client.

Example:

```
const express = require("express");
const app = express();

app.get("/divide", (req, res, next) => {
  try {
    let a = 10;
    let b = 0;

    if (b === 0) {
      throw new Error("Cannot divide by zero");
    }

    res.json({
      result: a / b
    });

  } catch (err) {
    next(err); // Pass the error to the error-handling middleware
  }
});

// Error-handling middleware
app.use((err, req, res, next) => {
  res.status(500).json({
    message: err.message
  });
});

app.listen(3000, () => {
  console.log("Server is running on port 3000");
});
```

Output: When you visit: **http://localhost:3000/divide**

Response:

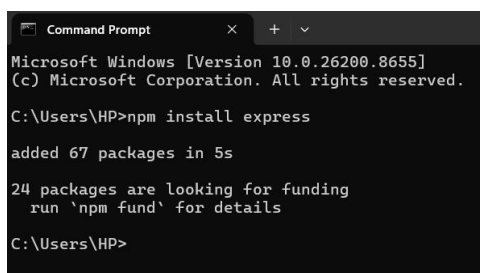
```
{  
  "message": "Cannot divide by zero"  
}
```

5.Explain NPM Package Management in detail. Discuss its features and commands

npm is originally an abbreviation for **Node Package Manager**. It is the default package manager and the world's largest software registry for the JavaScript and Node.js ecosystems.

npm is free to use. You can download all npm public software packages without any registration or login.

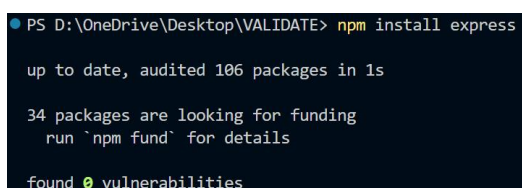
npm includes a **CLI** (Command Line Client) that can be used to download and install software:



```
Command Prompt  
Microsoft Windows [Version 10.0.26200.8655]  
(c) Microsoft Corporation. All rights reserved.  
C:\Users\HP>npm install express  
added 67 packages in 5s  
24 packages are looking for funding  
  run 'npm fund' for details  
C:\Users\HP>
```

NPM is the default Node.js package manager and a command-line tool used to install, manage, and share JavaScript packages.

- Manages dependencies and package versions.
- Supports installing, updating, and removing libraries.
- Acts as a public package registry.
- Enables package publishing and reuse.
- Simplifies automation and project maintenance.



```
PS D:\OneDrive\Desktop\VALIDATE> npm install express  
up to date, audited 106 packages in 1s  
34 packages are looking for funding  
  run 'npm fund' for details  
found 0 vulnerabilities
```

Features of npm

npm provides core tools and files that help manage packages, dependencies, and configuration efficiently in Node.js projects.

- **npm CLI:** The command-line tool used to interact with npm for installing packages, managing dependencies, running scripts, and publishing packages.
- **package.json File:** The project's main configuration file that stores metadata, dependencies, and npm scripts required to run the application.

- **package-lock.json File:** Records the exact versions of installed dependencies to ensure consistent and conflict-free installations across environments.

Basic npm Commands

Here are some commonly used npm commands:

- **npm init:** Initializes a new Node.js project and creates a package.json file.
- **npm install <package-name>:** Installs a package and its dependencies.
- **npm update:** Updates installed packages based on version ranges in package.json.
- **npm uninstall <package-name>:** Removes a package from the project.
- **npm run <script>:** Executes a script defined in the scripts section of package.json.
- **npm publish:** Publishes a package to the npm registry.
- **npm -v:** Check npm version.

6.Explain briefly about Request and Response Objects in Express.js

In Express.js, the **Request object (req)** represents the incoming HTTP request from the client, while the **Response object (res)** represents the HTTP response that the server sends back. These two objects are passed as arguments to your route handlers and middleware functions to manage the application's data flow.

1.req (Request) Object: The **req object** represents the incoming HTTP request sent by the client. It holds critical metadata, user inputs, and network information.

Property	What it contains	Example URL / Data	Example Value
req.body	Data sent in the request body	JSON body { "name": "John" }	{ name: "John" }
req.params	Dynamic values in the URL path	/student/101 with route /student/:id	{ id: "101" }
req.query	Values after ? in the URL	/search?name=John	{ name: "John" }
req.headers	Request metadata	Browser/Postman headers	{ host: "...", user-agent:"..." }
req.method	HTTP method used	GET, POST, PUT, DELETE	"POST"

2. res (Response) Object: The **res object** represents the HTTP response that the Express application sends back to the client. It provides custom methods to format, configure, and terminate the communication loop.

Method	Purpose	Example
res.send()	Sends a general response (text, HTML, object, buffer)	res.send("Hello")
res.json()	Sends a JSON response with the correct Content-Type header	res.json({ name: "John" })
res.status()	Sets the HTTP status code; usually chained with another response method	res.status(404).send("Not Found")
res.redirect()	Redirects the client to another URL or route	res.redirect("/home")
res.sendFile()	Sends a file (such as an HTML page, PDF, or image) to the client	res.sendFile(path.join(__dirname, "index.html"))